

VAE/vae.py

```
1 import torch
2 import torch.nn as nn
3 from torch import Tensor
4 from torch.nn import functional as F
5
6
7 class VAE(nn.Module):
8     def __init__(self, input_size: int, latent_size: int = 15):
9         super().__init__()
10        self.input_size = input_size # H*W
11        self.latent_size = latent_size # Z
12        self.hidden_dim = None # H_d
13        self.encoder = None
14        self.mu_layer = None
15        self.logvar_layer = None
16        self.decoder = None
17
18 #####
19         # TODO: Implement the encoder and decoder architectures described in the notebook.
20         #
21         #
22         # Encoder: self.encoder maps (N, 1, H, W) → (N, H_d) via nn.Sequential.
23         # Then self.mu_layer and self.logvar_layer each map (N, H_d) → (N, Z).
24         # Set self.hidden_dim = 400.
25         #
26         #
27         # Decoder: self.decoder maps (N, Z) → (N, 1, H, W) via nn.Sequential.
28         # Use nn.Unflatten to reshape the output back to image dimensions.
29         #
30 #####
31         # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
32
33        self.hidden_dim = 400
34        self.encoder = nn.Sequential(
35            nn.Flatten(),
36            nn.Linear(self.input_size, self.hidden_dim),
37            nn.ReLU(),
38            nn.Linear(self.hidden_dim, self.hidden_dim),
39            nn.ReLU(),
40            nn.Linear(self.hidden_dim, self.hidden_dim),
41            nn.ReLU()
42        )
```

```

40
41 self.mu_layer = nn.Linear(self.hidden_dim, self.latent_size)
42 self.logvar_layer = nn.Linear(self.hidden_dim, self.latent_size)
43
44 self.decoder = nn.Sequential(
45     nn.Linear(self.latent_size, self.hidden_dim),
46     nn.ReLU(),
47     nn.Linear(self.hidden_dim, self.hidden_dim),
48     nn.ReLU(),
49     nn.Linear(self.hidden_dim, self.hidden_dim),
50     nn.ReLU(),
51     nn.Linear(self.hidden_dim, self.input_size),
52     nn.Sigmoid(),
53     nn.Unflatten(1, (1, 28, 28))
54 )
55
56 # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
57
58 #####
59 #                                     END OF YOUR CODE
60 #
61 #####
62
63 def forward(self, x: Tensor) -> tuple[Tensor, Tensor, Tensor]:
64     """
65     Performs forward pass through FC-VAE model by passing image through
66     encoder, reparametrize trick, and decoder models.
67
68     Inputs:
69     - x: Batch of input images of shape (N, 1, H, W)
70
71     Returns:
72     - x_hat: Reconstructed input data of shape (N, 1, H, W)
73     - mu: Estimated posterior mu of shape (N, Z)
74     - logvar: Estimated variance in log-space of shape (N, Z)
75     """
76     x_hat = None
77     mu = None
78     logvar = None
79
80 #####
81 # TODO: Implement the forward pass:
82 #
83 # (1) Pass x through self.encoder, then self.mu_layer and self.logvar_layer
84 #
85 # (2) Call reparametrize(mu, logvar) to get the latent vector z
86 #
87 # (3) Pass z through self.decoder to reconstruct x
88 #

```

```

82 #####
83 # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
84
85 x = self.encoder(x)
86 mu = self.mu_layer(x)
87 logvar = self.logvar_layer(x)
88 z = reparametrize(mu, logvar)
89 x_hat = self.decoder(z)
90
91 # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
92 #####
93 #                                     END OF YOUR CODE
94 #
95 #####
96 return x_hat, mu, logvar
97
98 class CVAE(nn.Module):
99     def __init__(self, input_size: int, num_classes: int = 10, latent_size: int = 15):
100         super().__init__()
101         self.input_size = input_size # H*W
102         self.latent_size = latent_size # Z
103         self.num_classes = num_classes # K
104         self.hidden_dim = None # H_d
105         self.encoder = None
106         self.mu_layer = None
107         self.logvar_layer = None
108         self.decoder = None
109
110 #####
111 # TODO: Same architecture as VAE, but:
112 #
113 # - Encoder input size is (H*W + K): the flattened image concatenated with one-hot
label #
113 # - Decoder input size is (Z + K): the latent vector concatenated with one-hot label
#
114 # The flatten + cat operations happen in forward(), not here.
#
115 #####
116 # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
117
118 self.hidden_dim = 400
119 side = int(self.input_size*0.5)
120
121 self.encoder = nn.Sequential(
122     nn.Linear(self.input_size + self.num_classes, self.hidden_dim),

```

```

123         nn.ReLU(),
124     )
125
126     self.mu_layer = nn.Linear(self.hidden_dim, self.latent_size)
127     self.logvar_layer = nn.Linear(self.hidden_dim, self.latent_size)
128
129     self.decoder = nn.Sequential(
130         nn.Linear(self.latent_size + self.num_classes, self.hidden_dim),
131         nn.ReLU(),
132         nn.Linear(self.hidden_dim, self.input_size),
133         nn.Sigmoid(),
134         nn.Unflatten(1, (1, side, side)),
135     )
136
137     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
138
139 #####
140 #                                     END OF YOUR CODE
141 #
142 #####
143
144 def forward(self, x: Tensor, labels: Tensor) -> tuple[Tensor, Tensor, Tensor]:
145     """
146     Performs forward pass through FC-CVAE model.
147
148     Inputs:
149     - x: Input data of shape (N, 1, H, W)
150     - labels: One-hot class vector of shape (N, K)
151
152     Returns:
153     - x_hat: Reconstructed input data of shape (N, 1, H, W)
154     - mu: Estimated posterior mu of shape (N, Z)
155     - logvar: Estimated variance in log-space of shape (N, Z)
156     """
157     x_hat = None
158     mu = None
159     logvar = None
160
161 #####
162 # TODO: Implement the forward pass:
163 #
164 # (1) Flatten x and cat with labels, pass through encoder → mu, logvar
165 #
166 # (2) Reparametrize to get z
167 #
168 # (3) Cat z with labels, pass through decoder → x_hat
169 #
170 #####
171 # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

```

```

165
166     x_flattened = x.view(x.size(0), -1)
167     encoder_input = torch.cat((x_flattened, labels), dim=1)
168
169     hidden = self.encoder(encoder_input)
170
171     mu = self.mu_layer(hidden)
172     logvar = self.logvar_layer(hidden)
173
174     z = reparametrize(mu, logvar)
175     decoder_input = torch.cat((z, labels), dim=1)
176
177     x_hat = self.decoder(decoder_input)
178
179     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
180
181 #####
182 #                                     END OF YOUR CODE
183 #
184 #####
185
186     return x_hat, mu, logvar
187
188 def reparametrize(mu: Tensor, logvar: Tensor) -> Tensor:
189     """
190     Differentiably sample random Gaussian data with specified mean and variance
191     using the reparameterization trick.
192
193     We sample  $\epsilon \sim N(0, 1)$  and return  $z = \mu + \sigma * \epsilon$ , where
194      $\sigma = \exp(0.5 * \logvar)$ . This lets gradients flow through  $z$  to  $\mu$  and  $\logvar$ .
195
196     Inputs:
197     - mu: Tensor of shape (N, Z) giving means
198     - logvar: Tensor of shape (N, Z) giving log-variances
199
200     Returns:
201     - z: Tensor of shape (N, Z), each  $z[i,j] \sim N(\mu[i,j], \exp(\logvar[i,j]))$ 
202
203     HINT: Use torch.randn_like(mu) to sample epsilon – it automatically matches
204     the device and dtype of mu so you don't need to move anything to GPU manually.
205     """
206     z = None
207     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
208
209     epsilon = torch.randn_like(mu)
210     sigma = torch.exp(0.5 * logvar)
211     z = mu + sigma * epsilon
212
213     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
214     return z

```

```

213
214
215 def loss_function(x_hat: Tensor, x: Tensor, mu: Tensor, logvar: Tensor) -> Tensor:
216     """
217     Computes the negative variational lower bound loss term of the VAE.
218
219     Loss = reconstruction_loss + KL_divergence, averaged over the batch.
220
221     Reconstruction loss: binary cross entropy between x_hat and x.
222     KL divergence (closed form for diagonal Gaussian vs standard normal):
223         -0.5 * sum(1 + logvar - mu^2 - exp(logvar))
224
225     Inputs:
226     - x_hat: Reconstructed input data of shape (N, 1, H, W)
227     - x: Original input data of shape (N, 1, H, W)
228     - mu: Posterior mean of shape (N, Z)
229     - logvar: Posterior log-variance of shape (N, Z)
230
231     Returns:
232     - loss: Scalar tensor containing the mean loss over the batch.
233
234     HINT: Use F.binary_cross_entropy(x_hat, x, reduction='sum') for the
235     reconstruction term, then divide the total loss by batch size (mu.shape[0]).
236     """
237     loss = None
238     # *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
239
240     batch_size = mu.shape[0]
241
242     reconstruction_loss = F.binary_cross_entropy(x_hat, x, reduction='sum')
243
244     kl_divergence = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
245
246     loss = (reconstruction_loss + kl_divergence) / batch_size
247
248
249     # *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****
250     return loss
251

```